Model-driven Digital Twins for AECO

Philipp Zech*, Simon Senoner*, Emanuele Goldin*, Christoph Zallinger*, Sascha Hammes†, Judith Michael‡
*Department of Computer Science, University of Innsbruck, Austria
†Unit of Energy Efficient Buildings, University of Innsbruck, Austria
‡University of Regensburg, Programming and Software Engineering, Germany
Emails: {firstname.lastname}@uibk.ac.at, judith.michael@ur.de

***Abstract*—Digital Twins (DTs) are becoming central to the architecture, engineering, construction and operations (AECO) domain, enabling real-time monitoring, simulation, and life cycle-encompassing optimization of building operations through unified virtual representations of physical assets. However, practical deployment remains challenging due to fragmented data sources, domain-specific engineering workflows, and the absence of standardized integration frameworks. To address these issues, we present a model-driven framework for scalable DT deployment, grounded in Building Information Modeling (BIM). Our approach automates the extraction of domain semantics and the generation of runtime interfaces, thereby reducing manual effort and enhancing maintainability. The framework integrates Eclipse Hono and Ditto for real-time data exchange and leverages Unreal Engine for rich, interactive visualization. It also features standardized extension points to enable modular, service-oriented DT platforms. We demonstrate the practical application of our approach in a Living Lab case study and evaluate the entanglement of the resulting DT using the Overall Digital Twin Entanglement (ODTE) metric, underscoring its interoperability and architectural soundness.**

***Index Terms*—Model-driven Engineering, Model-driven Deployment, Building Information Modeling, Digital Twins, Unreal Engine**

I. Introduction

Digital Twins (DT) are cyber-physical information constructs that through carefully designed integration of static models with runtime data creates a virtual representation of a (cyber-)physical asset (e.g., a building) for simulation, analysis, and optimization [1], [2]. Crucially, in the architecture, engineering, construction and operations (AECO) domain, DTs enable data-driven infrastructure management and optimization by providing real-time insights, predictive analytics, and enhanced decision-making capabilities [3]–[7]. This transformation promises tangible benefits such as reduced operational costs, enhanced decision-making, and improved stakeholder collaboration along the complete building life cycle, from initial design to refurbishments and adaptations during operation [2], [8]–[12]. However, the realization of this potential substantially hinges on the ability to systematically and efficiently deploy DTs in complex and heterogeneous infrastructure environments [11]–[13].

Despite their promise, the deployment of infrastructure DTs remains elusive in practice [2]. This is primarily due to the fragmented nature of models and data, domain-specific engineering workflows, and a lack of standardized integration mechanisms [7], [14]. These challenges lead to highly customized and siloed solutions that are difficult to scale or replicate across projects. Without a cohesive deployment workflow, the AECO domain faces a bottleneck where the cost and complexity of DT implementations outweigh the perceived benefits, thus stifling broader adoption and DT-driven innovation [2], [7].

This raises a pressing question: *how can we deploy maintainable and interoperable DTs in AECO at scale, despite complications from heterogeneous models, fragmented data landscapes, and entrenched engineering practices?* As the construction industry stands at the intersection of digital innovation and practical implementation, answering this question is critical for unlocking the full potential of DTs and driving meaningful, domain-wide transformation and innovation.

In this paper, we propose a model-driven deployment framework that leverages Building Information Modeling (BIM) as a semantic backbone for automating the creation, configuration, and deployment of DTs in AECO. Our approach systematically extracts relevant asset data from a BIM model and generates runtime interfaces for bidirectional communication. This enables end-to-end automation of the DT pipeline — apart from initial building modeling and building-site device configuration — thus significantly reducing manual effort. By integrating with Eclipse Hono [15] and Ditto [16], our proposal supports real-time data exchange, while Unreal Engine 5 [17] provides a rich visualization environment for interactive monitoring and simulation. Crucially, our platform features standardized extension points for modular service integration, supporting scalable deployment and future extensibility. We demonstrate the practical viability of our proposal through a DT of a Living Lab, and by measuring how well the resulting DT reflects its physical counterpart using the Overall Digital Twin Entanglement (ODTE) metric [18].

Organization. Section II outlines the research context of our work. Section III defines BIM's role and introduces requirements for model-driven deployment framework for DTs. Section IV details our solution. Section V presents the case study and evaluation, and discusses results and limitations. Section VI concludes our paper.

II. Research Context and Problem

The work presented in this paper is rooted in two ongoing research projects, viz. TwinLight [19] and BOREALIS [20], which together establish the research context for model-driven

This research has received funding from the Austrian Research Promotion Agency (FFG) under Grant Agreement No.: 898708, TwinLight, and Grant Agreement No.: 910225, BOREALIS.

DT deployment in the built environment. Both projects explore how runtime semantics, such as sensor configurations, control logic, and live operational data can be systematically embedded into BIM models, transforming them from static planning artifacts into dynamic, runtime-capable digital representations, cf. DTs.

TwinLight addresses the fragmentation and opacity of current building automation workflows by introducing a BIM-centric control configuration framework. This framework aims to improve system transparency and interoperability across construction phases and stakeholders by embedding domain-specific control semantics for lighting systems directly into BIM models. To this end, TwinLight proposes a layered system architecture that couples BIM with a middleware for automated commissioning, bidirectional communication between virtual and physical assets, and seamless integration of non-trivial, user-centered control strategies [9], [21].

BOREALIS, in contrast, focuses on adaptive lighting systems using reinforcement learning to balance and optimize energy consumption, visual comfort, and non-visual health effects. Its core innovation lies in using post-occupancy sensor data to iteratively improve control policies over time, thus closing performance gaps between design and operation. Here, BIM serves not only as the model backbone for spatial and semantic data integration, but also as a "digital RL environment" where control models and sensor feedback converge. BOREALIS exemplifies how runtime aspects can be both engineered and evolved within a BIM-based DT ecosystem.

Together, these projects provide a practical motivation for this paper's proposal: a model-driven deployment framework that bridges the gap between BIM modeling and the operational realities of infrastructure DTs.

A. Related Work

DTs are increasingly recognized in the AECO domain as to their potential in enabling real-time synchronization between physical assets and their digital counterparts. Recent research highlights the integration of DTs with BIM, artificial intelligence, and sensor networks to enhance life cycle management, cost modeling, and operational efficiency [7], [14], [22]–[24]. Current literature emphasizes the role of modeling and workflow automation in achieving consistent and scalable DT implementations, particularly for modular construction and infrastructure monitoring [6], [13], [25], [26]. Despite its promise, DT adoption remains limited by fragmented, case-specific deployments that often lack standardized data schemas, robust integration frameworks, and bidirectional communication protocols [7], [22], [25], [27]. These technical challenges are compounded by organizational barriers, including insufficient stakeholder alignment, high implementation costs, and the absence of unified engineering workflows [23], [28], [29]. Addressing these limitations will require more than technological advancement by demanding a rethinking of system and tool integration, stakeholder collaboration, and governance across the asset life cycle [13].

As for open source DT frameworks, Gil et al. [30] identify 14 frameworks, most of which target industrial or IoT applications and offer infrastructure-level components rather than complete, deployable systems. While frameworks like Eclipse Ditto [16] and BaSyx [31] provide robust connectivity and standard compliance, they require significant manual configuration, integration of analytics, and orchestration of components. Out-of-the-box functionality is generally limited, and usability for non-developers is low.

In the AECO sector, the situation is more restricted [2], [11]. Most frameworks are not designed with AECO-specific standards like BIM in mind, and those that can be adapted — such as Bentley's iTwin.js [32] — rely on proprietary ecosystems or require substantial manual effort due to a lack of automation.

Recently, Gerber et al. [33] presented an ad-hoc DT for a Living Lab for a LEED Platinum academic building based on BIM for monitoring, simulation, and performance analysis. Their study emphasizes automated data acquisition, normalization, and 3D visualization, while highlighting challenges in data integration and stakeholder coordination, insights that directly inform model-driven approaches in DT engineering. Their results reinforce earlier findings by Opoku et al. [34] who also demonstrated the effective integration of DTs with BIM and IoT platforms to optimize indoor environmental conditions in a university library setting. Both studies underscore the significance of real-time data utilization and visualization in enhancing facility management and highlight common challenges in data integration and stakeholder engagement in DT implementations. In synopsis, AECO adoption is dominated by commercial tools or custom-built solutions [2], [13]. Notably, MDE is not present across the surveyed works, leaving a critical gap in abstraction and reuse capabilities essential for complex domains like AECO.

Model-driven engineering (MDE) offers a streamlined approach to handling the increasing complexity of systems by elevating models to primary engineering artifacts [35], [36]. It enables abstraction, formalization, and automation across the development life cycle, reducing cognitive overload and improving consistency in large-scale systems [37]. MDE supports the definition of domain-specific modeling languages and metamodels that unify engineering workflows, providing a structured foundation for communication across disciplines and life cycle phases [38], [39]. Importantly, MDE also strengthens governance by encoding organizational policies and architectural decisions into formal models, promoting compliance, reuse, and quality assurance [40]. Recent advances continue to refine the integration of MDE into DT systems [41]–[47]. As reported in Lehner et al.'s recent survey, research primarily focuses on runtime adaptability, semantic coherence, and tool interoperability [48]. Abbasi et al. [49] examined the phenomenon of *semantic drift*, where the alignment between digital models and their physical counterparts diverges over time. Their study stresses the necessity of synchronization techniques to ensure the integrity of model-based DTs. Kamburjan et al. [50] proposed a declarative life cycle specification language, enabling dynamic evolution and rule-based management of DT instances. Meanwhile, Oakes et al. [51] advanced ontological frameworks to support service-

oriented DT architectures, improving semantic integration across platforms. Giulianelli et al. [52] introduced a web-based approach to engineer interoperable ecosystems of DTs, facilitating seamless integration and communication across diverse platforms. These contributions underscore the evolving landscape of DT engineering, where MDE-inspired methodologies are pivotal in managing both growing complexity and dynamic behaviors in cyber-physical systems [53].

Recently, the application of MDE for engineering DTs of buildings has received first attention [9], [11], [12], [21], [54]–[56]. Specifically, the integration of BIM with MDE for automating the deployment of DTs for buildings has been proposed [9], [11], [12], [21], [56], [57]. These approaches show promising results by directly leveraging BIM models for model-to-model and model-to-text based transformations [58] for generating runtime artifacts for the (semi-)automatic establishment of DTs. The current work aims at streamlining these results into a BIM-based, model-driven framework for the automated deployment of DTs of building infrastructures.

B. Challenges and Contributions

Despite increasing interest in, and ongoing research efforts on DTs in the AECO domain, several systemic challenges (**C***X*) hinder their widespread deployment as elaborated so far:

- **C1:** BIM data is fragmented and semantically heterogeneous, hindering unified interpretation across disciplines.
- **C2:** DT deployment remains largely manual, resulting in inconsistent setups and high engineering overhead.
- **C3:** Existing DT frameworks lack seamless, real-time interaction between virtual models and physical systems.
- **C4:** Current DT implementations offer limited extensibility, making it difficult to integrate or evolve services.

In response (**R***X*) to these challenges, this work introduces a model-driven DT deployment framework grounded in BIM, which advances the state-of-the-art through the following contributions:

- **R1:** A modeling methodology to standardize BIM semantics for a model-driven transformation pipeline to generate consistent runtime artifacts (**C1**).
- **R2:** An automated deployment process that reduces manual intervention and enables reproducible DT setups (**C2**).
- **R3:** Real-time data integration using Eclipse Hono and Ditto, enabling live synchronization between physical assets and digital twins (**C3**).
- **R4:** A modular architecture with defined extension points to support scalable, service-oriented DT development in AECO (**C4**).

Together, these contributions aim to operationalize scalable and interoperable DTs in AECO contexts. By closing the gap between design-time models and runtime operations, we offer a sustainable path toward domain-wide DT adoption.

Our work is organized using Design Science Research (DSR) [59] and provides artifacts to facilitate model-driven deployment of operational control DTs in the AECO domain. We start with requirements elicitation (cf. Section III-A) and conclude with prototyping and experimentation (cf. Section V). Our artifacts are developed in response to the following design science problem [59]:

Improve	*DT deployment in BIM-based infrastructure projects* (context)
by designing	*a model-driven automation framework* (artifact)
that satisfies	*semantic integration, real-time connectivity, and extensibility* (requirement)
in order to	*enable scalable DT engineering in the AECO domain.* (goal)

III. BIM-based Digital Twins

BIM is increasingly regarded as the principal modelling paradigm in the AECO sector, providing a comprehensive framework for generating, managing, and disseminating digital building information throughout the entire building life cycle with policy initiatives and regulatory mandates decisively fostering and accelerating this development [60]. BIM enables the creation of semantically rich, object-based 3D models that incorporate geometric, spatial, and functional characteristics of building components and their relationships. These models provide a collaborative platform where architects, engineers, and construction professionals can coordinate design decisions, detect clashes, simulate performance, and generate documentation [61].

While there are also DTs and industry-specific solutions for the digital mapping of assets in other industries, such as the manufacturing industry (e.g. capitalizing on Model-based Systems Engineering; MBSE) [1], [62], BIM is becoming the standard solution in the construction industry [63], [64]. Requirements regarding the complexity of construction projects and availability over long building life cycles are covered by BIM. It is therefore essential that concepts such as DTs in the construction sector are also designed for longevity and compatibility with existing, AECO-specific systems and standards such as BIM.

Despite these strengths, BIM however exhibits significant limitations when considering its application as a foundation for DTs [7], [9], [14]. First, BIM models are predominantly static representations focused on design and construction phases, lacking mechanisms for real-time data integration from operational buildings. Second, BIM tools typically provide limited support for bidirectional communication with physical systems, a core requirement for functional DTs. Third, while BIM excels at representing physical and spatial aspects of buildings, it lacks native capabilities for modeling runtime aspects of Building Control Systems (BCS), essential for operational monitoring and control [7], [11], [12], [21].

These limitations create a fundamental gap between BIM's design-time focus and the operational requirements of DTs. DTs require continuous synchronization with physical assets through sensor data, the ability to process and analyze this data in real-time, and mechanisms to feed control signals back to the BCS. In addition, DTs also require extensible architectures that can accommodate evolving services and analytics throughout the building life cycle [7].

Nevertheless, BIM provides an ideal semantic foundation for DTs due to its comprehensive representation of building

geometry, topology, and component relationships. What is needed is a systematic approach to transform these static BIM models into dynamic, operational DTs by addressing the identified limitations and extending BIM through a formal methodology for modeling BCS runtime aspects and their integration with a building's digital representation.

A. Uses Cases, Requirements, and Solution Proposal

TABLE I: Actor Roles in our Model-driven DT Deployment Framework

Role	Description
Building Control Engineer	Models and configures devices, defines the control logic, and initiates DT deployment from BIM models.
Facility Manager	Monitors system states, interacts with the twin, and operates the physical infrastructure via the DT.
DT Engineer	Develops and integrates custom services and plugins to extend DT functionality.

Commensurate with the limitations of BIM as a seamless enabler for DTs, we have developed a comprehensive set of requirements for our model-driven deployment framework. These requirements emerge from both theoretical considerations and practical insights gained through our research projects, TwinLight and BOREALIS (cf. Section II).

To address the gap between BIM's static representations and the dynamic nature of DTs, our proposal must support the complete life cycle of DT deployment and operation. Table II outlines the use cases to be supported by our framework, spanning from initial device modeling to the development of custom DT services atop our proposal. These use cases reflect the core functionality needed by different stakeholders (cf. Table I) involved in the DT life cycle, including *Building Control Engineers* who model devices and implement control logic, *Facility Managers* who monitor and control building operations, and *DT Engineers* who extend functionality through custom services.

The functional requirements (**F***X*) detailed in Table III form the conceptual foundation of our solution, beginning with device modeling capabilities that address BIM's fundamental limitation in representing BCS components. This enables building control engineers to define sensors, actuators, and controllers directly within the BIM environment, creating a unified semantic foundation for subsequent transformation steps. The framework then leverages MDE principles to extract structured representations from these enriched BIM models and automatically generate deployable DT artifacts with minimal manual intervention.

Real-time data exchange capabilities ensure continuous synchronization between physical buildings and their digital counterparts, while interactive visualization provides immersive interfaces for monitoring and control. The framework's modular architecture, with standardized extension points, supports the integration of custom services and analytics, allowing DTs to evolve with changing operational requirements. Runtime configuration management further enhances adaptability by enabling adjustments without requiring full redeployment.

Complementing these functional requirements, our non-functional requirements (**NF***X*) address critical aspects of interoperability across heterogeneous systems, scalability from individual components to entire infrastructure systems, maintainability through low-overhead updates, and accessibility for users with varying technical expertise.

This requirements-driven approach provides a systematic foundation for implementing DTs that bridge the gap between BIM's static representations and the dynamic operational needs of the AECO domain, addressing the challenges identified in Section II. At this, we want to emphasize that these are not all requirements, but rather those pertinent to crafting a technological spike solution tailored to the specific challenges and innovations explored in this work (cf. Section V-C). The discussion section will revisit these requirements to evaluate their effectiveness and role in advancing the proposed solution.

B. Solution Proposal

Figure 1 (p. 6) presents the conceptual model of our framework, illustrating how the requirements from Table III are implemented through coordinated roles and system components. Figure 1 demonstrates the flow from enriched BIM models to operational DTs, emphasizing the key roles and their contributions to the DT life cycle.

Building Control Engineers, using `BIM tools`, perform device modeling and initiate the deployment process. *Facility Managers* then interact with the deployed DT for real-time monitoring and control. *DT Engineers* extend the framework's capabilities through service integration. The `DT Framework` acts as a central hub, facilitating bidirectional data exchange and enabling a holistic approach to building life cycle management.

IV. Model-based Deployment of Digital Twins

Figure 2 (p. 7) outlines our solution proposal. Starting with a BIM model which contains all relevant information pertaining deployment, e.g., modeled devices and relevant middleware metadata for Hono and Ditto (cf. Section IV-A for a detailed discussion of the modeling and deployment methodology), after validating the model [65], the model is fed into the `Hono/Ditto Configurator` and scanned for "twinnable" devices to extract relevant metadata (e.g., tenant and device ID or endpoint addresses) for Hono and Ditto. The extracted metadata is then used to automatically configure Hono and Ditto for both receiving values from devices but also sending data (cf. commands) down to receiving devices in a building. At the same time, the BIM model is fed into Datasmith [66] to convert the BIM model into UE5's internal representation (as part of the `Model Converter`). Once Hono and Ditto are configured, UE5 receives initial device values and parameters via an HTTP POST request and subsequently connects to Ditto via a Websocket connection for both retrieving data from and sending commands to devices, respectively. In the event of commands, we wait for a confirmation of its execution before we update the DT values (e.g., the state of a switch). As

TABLE II: Use Cases for our Model-driven DT Deployment Framework.

ID	Name	Description
UC1	Model BCS Devices	*Building Control Engineers* define and embed BCS devices, including their parameters and metadata (cf. runtime aspects), directly into the BIM model.
UC2	Deploy DT Instance	*Building Control Engineers* automatically deploy a DT instance using enriched BIM data, with minimal manual intervention.
UC3	Monitor Asset Status in Real-Time	*Facility Managers* observe live sensor data and system states from the physical twin through the DT interface, facilitating real-time monitoring.
UC4	Send Commands to Physical Assets	*Facility Managers* send commands from the DT interface to control physical infrastructure components, via bidirectional communication.
UC5	Visualize and Interact with Twin	*Facility Managers* use interactive 3D views of the DT to navigate, inspect, and interact intuitively with the building.
UC6	Extend DT with Custom Services	*DT Engineers* use API- and message-based interfaces to integrate new analytical or control services with the DT after its deployment.

TABLE III: Functional (F*X*) and Non-Functional (NF*X*) Requirements for our Model-driven DT Deployment Framework

ID	Name	Description
F1	Device Modeling	Support modeling of BCS devices for subsequent BIM-based digital twinning.
F2	Semantic Model Extraction	Extract structured, domain-specific representations from BIM models.
F3	One-Click Deployment	Automate the generation, configuration, and launch of DTs from enriched BIM inputs.
F4	Live Data Exchange	Enable real-time, bidirectional data flow between the digital and physical twins through middleware platforms.
F5	Interactive Visualization	Render interactive DT environments using 3D visualization frameworks.
F6	Modular Service Integration	Allow extension of DT functionalities via a standardized interfaces.
F7	Runtime Configuration Management	Support dynamic configuration of deployed DTs, including interface updates and parameter tuning.
NF1	Interoperability	Ensure compatibility across heterogeneous BIM tools, IoT protocols, and visualization systems.
NF2	Scalability	Support a wide range of use cases from individual building components to entire infrastructure systems.
NF3	Maintainability	Facilitate low-overhead updates, service isolation, and life cycle traceability.
NF4	Accessibility	Provide intuitive interfaces for users with varying technical expertise.

to using Eclipse Hono and Ditto, we benefit from built-in retransmission and acknowledgement mechanisms that persist each message to reliable broker storage before sending an acknowledgement, giving us resilient at-least-once delivery even during brief network or service disruptions. The resulting visualizations in UE5 (cf. Section V) can be grouped by the most fine-grained level (depending on the input BIM model) and filtered, to e.g., only show a building up to a certain storey.

A. Modeling and Deployment Methodology

For preparing the BIM model with relevant metadata for modeled devices, we leverage Autodesk Revit by defining custom modeling families and property sets to model devices and their metadata [9], [21], [57]. When applied to an object in the BIM model, these families are interpreted as "twinnable" devices. Crucially, each device, among others, must provide an `Instance General UUID` as well as having its IFC-type set to the corresponding device type, e.g., `IFCLIGHTFIXTURETYPE` in the event of a luminaire. After all devices have been modeled, Revit is used to export the BIM model in IFC[1]. The resulting IFC BIM model can be uploaded to our platform either via drag-and-drop or a file chooser.

[1]Industry Foundation Classes, an open model and data exchange format for BIM [67].

Prior to uploading the model, we perform model checking to ensure that all relevant metadata is modeled for the devices. For this, we leverage a model checker specifically developed for IFC-based BIM models [65]. Listing 1 shows a sample rule used to check if an `Instance General UUID` is present.

```
[hasUuid :
    (?s rdf:type spec:Device)
    (?s spec:hasUuid ?u)
    strlen(?u) == 0
        -> (?s spec:check_UUID spec:FAIL)]
```

Listing 1: Example rule to check whether `Instance General UUID` is present for devices.

When loaded by the `Hono/Ditto Configurator`, the IFC model is scanned to identify "twinnable" devices by the presence of the IFC parameter `Instance General UUID` or any field containing a value starting with `utid:`. Following, for each device, we create the relevant configurations for creating a low-level device twin inside Ditto which then automatically connects to Hono for bidirectional device communication. The definition of these device twins follows the required format of Ditto to provide necessary attributes and features, where features represent dynamic values and attributes represent static information. Both types support arbitrary JSON values.

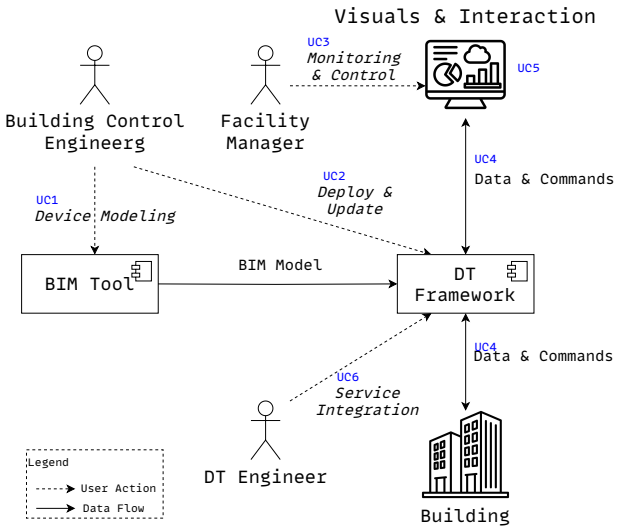


Fig. 1: Concept of our model-driven DT deployment framework (with use cases attached in blue). The diagram outlines the transformation of enriched BIM models into DT runtime artifacts through coordinated roles and system components. *Building Control Engineers* configure device semantics and control logic via the BIM tool; *DT Engineers* integrate and extend services; *Facility Managers* interact with deployed DTs for monitoring and control. Our proposal supports automated deployment, live data exchange, and bidirectional communication between digital and physical assets through an interactive visualization platform.

As for the renderings of our DT in UE5, as stated above, we use Datasmith to convert the IFC model into UE5's internal representation for scene rendering. In addition, the `Model Converter`, for each identified "twinnable" device in the IFC model, generates a UE5 component which is attached to the corresponding object in the resulting scene rendering for runtime interaction and visualization. Internally, all devices are indexed in a map using their `Instance General UUID` for fast access. As a central orchestrator, the `UnrealTwinController` — a custom developed event broker that manages communication with any `TwinComponent` (cf. a connected device) — is attached to the root of the scene rendering model and manages all devices.

On the technical side, Eclipse Hono and Ditto are deployed via Kubernetes [68] and communicate via Apache Kafka [69]. Both services are deployed and configured using Helm [70]. This setup is performed once during installation. Devices can then be registered with Hono and Ditto through HTTP requests (which we also exploit for automatically configuring both, Hono and Ditto, from the metadata in the BIM model). All device data is persisted in a MongoDB [71] instance deployed alongside Hono and Ditto to render device registration a one-time operation.

As mentioned at the beginning, our solution is implemented for Autodesk Revit. For other BIM tools, the identifier can be placed in any available field using the `utid:` definition which then will also be recognized by our solution as a "twinnable" device. Zech et al. [57], [58] provides a detailed discussion of the core ideas of the underlying modeling infrastructure. Regarding model evolution, Zech et al. [72], [73] provide a scalable approach to evolve models of a DT alongside its physical asset.

B. DT Services

As for dedicated DT services, our proposed solution already provides two crucial ones, viz., (i) 3D-based rendering and visualization of the twinned asset, and (ii) monitoring and control facilities, both provided via the interface of UE5. Specifically, the 3D representations is also used to visualize the current building state via, e.g., heatmaps. This allows visualizations of the distribution of values, for instance to show the temperature gradient within a space. In addition, our solution provides extension points which allow for the implementation of custom DT services.

At the communication layer, the use of Hono and Ditto is beneficial in that both provide standardized messaging interfaces (MQTT, AMQP) which can readily be subscribed and published to by additional services. In addition, the Websocket connection we maintain between UE5 and Ditto further can be piggybacked by additional services to not only interact with devices but also higher level components of our solution. In line with these "off-the-shelf" extension points, UE5 also provides a high-level plugin interface for developing custom services as plugins. Being fully integrated into UE5, these plugins readily can interact with any connected device and can be added without modifying our core components.

Finally, via the `TwinComponent` base class, our solution also provides a custom programmatic extension point to attach additional services to any "twinned" device. Figure 3 shows the underlying design of this extension point. By extending from the `TwinComponent` base class, custom services can be implemented at the device level in UE5.

One example is the `LightTwinComponent`, which is instantiated for all devices labeled as `IFCLIGHTFIXTURETYPE` in the IFC model. Specifically, the `LightTwinComponent` component adds a UE5 light to the associated 3D representation and interprets Revit-specific photometric values (provided via device metadata in the BIM model) for accurate light representation in the scene rendering. In addition to managing any `TwinComponent`, the `UnrealTwinController` brokers messages (e.g., device value updates or commands) between a component and the the corresponding "twinned" device via the `DittoController` which in turn manages the devices (cf. `DittoDevice`) and associated connections (cf. `DittoConnection`). Technically, the `UnrealTwinController` relies on UE5 *Delegates* (cf. function pointers) and implements the *Observable* pattern [74]. Via the `FeatureParserController` additional behavior for a component can be implemented. For the case of the `LightTwinComponent`, we have implemented an additional feature parser (cf. `LightFeatureParser`) which "reacts" to values such as `on`, `brightness`, and `color_temp_kelvin`

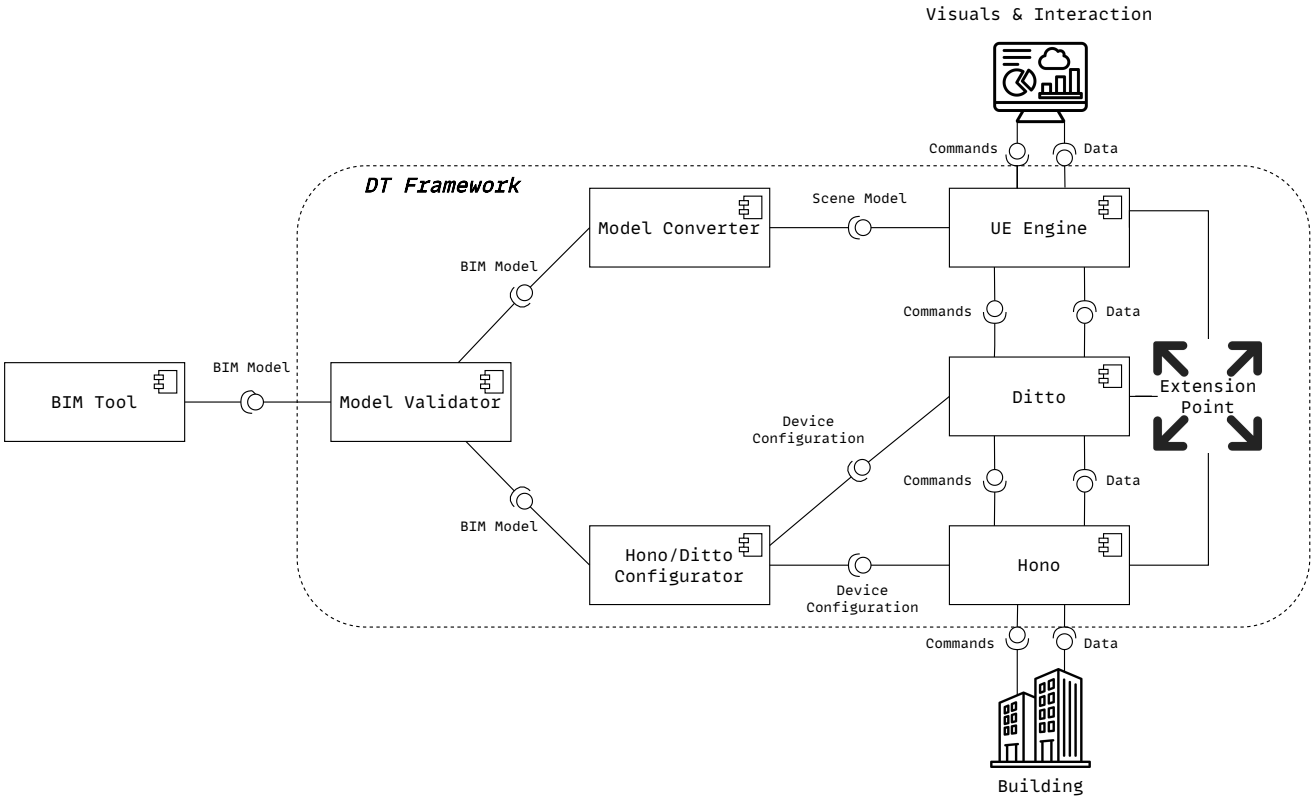


Fig. 2: Component and connector view of our model-driven DT deployment framework. The DT Framework processes enriched BIM models and device configurations through validation, conversion, and configuration components. These feed into middleware (Hono and Ditto) and visualization services (UE Engine), enabling bidirectional integration with the physical building and immersive interaction platforms. Extension points facilitate modular integration of additional services, supporting scalable, service-oriented twin deployments.

and updates the UE5 scene rendering accordingly. These programmatic extension points (cf. `TwinComponent` and `FeatureParser`) allow for extending our proposal with additional, custom DT services inside UE5. Observe that `HVACFeatureParser` and `HVACTwinComponent` in Figure 3 are only for illustrative purposes and currently not implemented in our proposal. Table IV provides a concise overview on these extension points.

Our framework ensures data integrity through Eclipse Ditto's policy-based validation, which enforces value ranges and units for sensor data, and pre-execution checks for actuator commands to prevent conflicts or unsafe operations. Fault tolerance is provided by Eclipse Hono's at-least-once message delivery and Ditto's persistent event storage, ensuring resilience against network failures or service disruptions. These mechanisms collectively maintain synchronization and operational safety between physical assets and their DTs.

V. Results and Discussion

We have evaluated our approach in the context of a Living Lab [75] at the University of Innsbruck. Figure 4a shows the BIM model of the Living Lab, which comprises an office space of $22\,\mathrm{m}^2$, two working places, as well as various sensors and devices for monitoring and controlling, among others, artificial and natural lighting. In Figure 4a, sensors are indicated with circles, rectangles denote luminaires, and rhomboids the controllers (e.g., of the artificial lighting systems). Table V shows an overview of devices installed in the Living Lab that are relevant for artificial lighting control.

Prior to deploying a DT from the BIM model, a translation script has been developed that allows to bridge the MQTT broker running in the Living Lab with Hono's MQTT broker. This script is deployed as part of the device setup process described earlier under Section IV-A which also handles device initialization using the configurations received from the `Hono/Ditto Configurator`. Observe that this script is a necessary fixture to connect two MQTT brokers to each other. In the event of devices directly exposing MQTT (or similar) endpoints, such a translation script is not necessary. All deployments and experiments were conducted on a Razer Blade 15 2021 base model laptop with an Intel® i7 11800H CPU, RTX™ 3070 MAX-Q GPU, and $64\,\mathrm{GB}$ memory.

A. Qualitative Evaluation

The qualitative evaluation centers on the deployment process of the DT within the Living Lab, emphasizing the

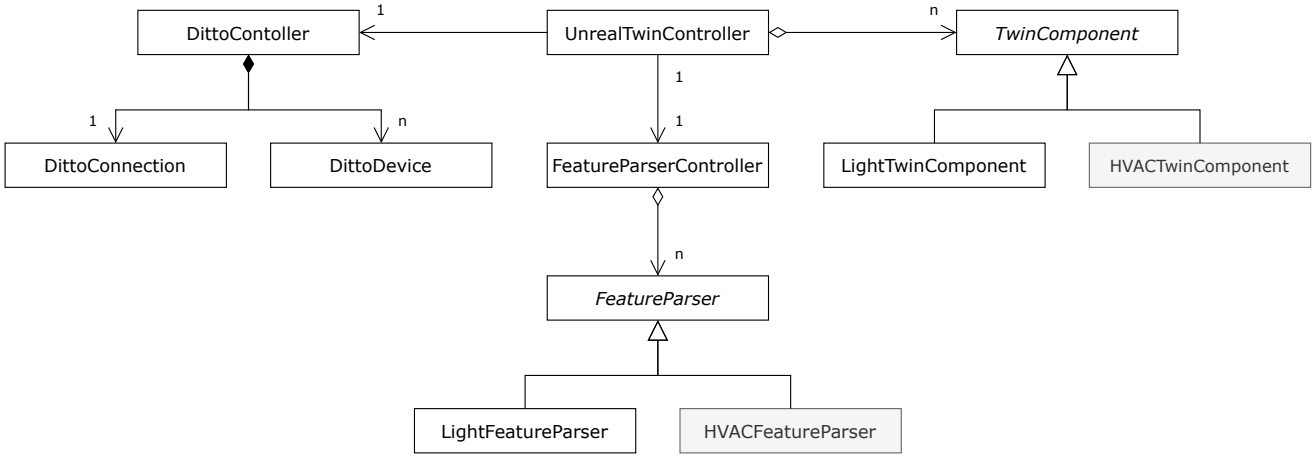


Fig. 3: Underlying design architecture of the `UnrealTwinController` and its managed `TwinComponents`. Custom services can be added at the device level by extending the `TwinComponent` and `FeatureParser` base classes (see examples for light or for Heating, Ventilation, Air Conditioning (HVAC); observe that the latter, e,.g., `HVAC*`, currently is not implemented in our proposal, but instead illustrates straight-forward extensibility to other building trades.).

TABLE IV: Extension points provided by our solutions proposal with example services.

Extension Point	**Mechanics**	**Example**
UE5 Plugin Interface	Provides a plugin mechanism to add custom DT services without modifying the core framework	Custom 3D visualization module
`TwinComponent` Extension	Allows implementing custom `TwinComponent` for device-level services	Device-specific control (e.g., light control)
`FeatureParser` Extension	Allows implementing custom `FeatureParser` for custom logic	Dynamic brightness adjustment based on sensor data
Communication Interface	Uses standard protocols (MQTT, AMQP) and Websockets for messaging and integration	Integration with third-party analytics over MQTT

TABLE V: Overview of devices installed in the Living Lab.

Illuminance sensors on table	TSL2591
Illuminance sensors on ceiling	BH1750
Occupancy sensor at the workplace and on ceiling	HC-SR501 PIR
Luminaires on ceiling	Ludwig Leuchten (each 2x T5 lamps of type 840 with 49W)
Lighting controller	Raspberry Pi 4 Model B, 4x 1,5 GHz, 8 GB RAM, WLAN, BT

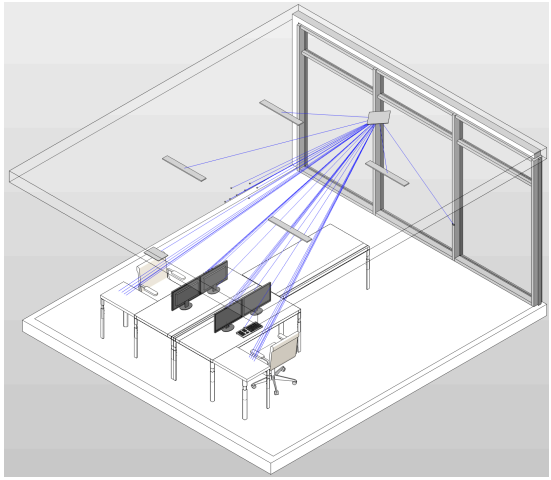
practical steps and outcomes. The modeling phase began with the enrichment of the BIM model by integrating devices related to monitoring and controlling (artificial) lighting as well as their relevant metadata for subsequent deployment and device configuration (cf. Section IV-A). The result of this modeling process was validated using our model checker [65] to ensure that each device was adequately represented and prepared for DT integration. Following the modeling, the deployment involved the seamless transition of this enriched model into the DT framework, facilitated by automated device configuration and connection setup. The result was a fully operational DT that dynamically mirrored the physical Living Lab environment as showcased in Figure 4b. This deployment not only demonstrated our solution's capability to automate complex processes but also its potential to enhance real-time monitoring and control. A video offering insights into our DT in action can be accessed through Zenodo [76].

B. Quantitative Evaluation

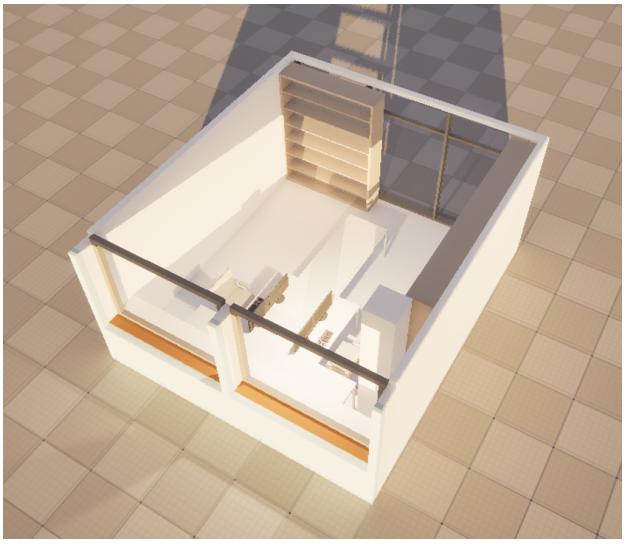
The quantitative evaluation of our proposal was conducted to assess the system's performance using precise metrics. This involved measuring deployment times and evaluating the system's synchronization and interoperability through the Overall Digital Twin Entanglement (ODTE) metric [18].

To assess the efficiency of our proposal, we measured the time taken for three critical stages of the deployment process: model checking, model parsing, and initialization and setup of the DT inside UE5 including configuration of Hono and Ditto. Table VI shows our results.

The model checking process, which verifies the enriched BIM model for completeness and accuracy takes around 5.53 seconds. The model parsing stage, involving model processing and configuration generation, averaged 8.7 seconds. Finally, the initialization and setup of the DT in UE5, where the model transitions into a fully operational state, was completed



(a) BIM model of the Living Lab used during our evaluation. Observe that this model hides some element, e.g., the book shelf as depicted in Figure 4b, to reduce clutter.



(b) Rendered Living Lab inside UE5.

Fig. 4: Visualization of the Living Lab: (a) BIM model in Revit (b) 3D rendering in UE5.

TABLE VI: Deployment Times for Digital Twin

Stage	**Duration** (seconds)
Model Checking	5.53
Model Parsing	8.7
Initialization and Setup	0.3

TABLE VII: ODTE Measurements

ODTE Component	**Score**
Timeliness	0.997
Reliability	1.0
Availability	1.0
Total Score	0.997

in 0.3 seconds. This considerably short deployment time of 14.53 seconds underscores the efficiency of our proposal in streamlining the DT creation process. Our deployment was done on a standard

The Overall Digital Twin Entanglement (ODTE) [18] metric is a crucial measure of the synchronization quality between DTs and their physical counterparts. It is composed of three main components: timeliness, reliability, and availability. *Timeliness* (T) reflects how up-to-date the data is that the DT receives, measuring the delay between a state change occurring in the physical counterpart and the subsequent update in the DT. *Reliability* (R) measures the consistency in communication between the physical counterpart and the DT, defined as the ratio of the number of state updates successfully received by the DT to the number of updates expected. *Availability* (A) assesses the operational uptime of the physical counterpart as perceived by the DT, measuring the extent to which the physical counterpart is accessible and capable of sending updates. The ODTE score then is calculated as the product of these components (cf. $ODTE = T \times R \times A$), providing a comprehensive assessment of system performance. Table VII shows our results.

In our assessment, the scores were 0.997 for timeliness (indicating some slight delay in communication), 1.0 for reliability (no message loss), and 1.0 for availability (100% up-time) resulting in an overall ODTE value of 0.997. We measured the ODTE metric over a continuous 12-minute measurement window. The setup employed a regular 1-second heartbeat mechanism to coordinate data exchange and state updates between the physical and digital twin. Throughout the experiment, the DT maintained a consistent and exact reflection of the physical system's state, with no observable divergence in state, reflecting ideal entanglement under the given conditions. Our experiments assess bidirectional synchronization by transmitting lighting control commands from the DT to the physical system and verifying the corresponding state updates in the reverse direction. The absence of such updates would indicate a communication failure, resulting in a total ODTE score of 0, thereby validating the integrity of the bidirectional communication channel.

Our experiment aligns with the scope outlined in Bellavista et al., who introduced the ODTE metric and demonstrated its applicability over comparable time windows in controlled environments [18]. While real-world deployments may introduce latency, jitter, or state drift, the controlled laboratory conditions and structured update cycles in our setup allowed for a theoretical maximum ODTE score to be reached and confirmed. This benchmark offers a reference point for evaluating partial entanglement in more complex or distributed deployments.

C. Discussion

The results of our evaluation demonstrate that our proposed model-driven DT deployment framework effectively addresses

the challenges outlined in Section II. The measured overall deployment duration (14.53, cf. Table VI) seconds validate that our approach can streamline processes that were traditionally ad-hoc, manual, and labor-intensive. In tandem with the ODTE score achieving nearly ideal synchronization (0.997, cf. Table VII), these findings confirm that our proposal provides the real-time consistency necessary for DT systems in socio-technological settings (cf. building operations) [77].

Our proposed approach effectively fulfills both functional and non-functional requirements (cf. Table III), ensuring the successful deployment of DTs as described in our use cases (cf. Table II). By providing a standardized and automated integration of BIM semantics, we achieve seamless deployment and facilitate real-time, bidirectional data exchange. This ensures that the DT not only mirrors the physical state accurately but also enhances interactive visualization and control through UE5. The integration of middleware components such as Eclipse Hono and Ditto underscores the robustness of our solution by leveraging standardized protocols like MQTT, ensuring interoperability and scalability in data transfer. As demonstrated in earlier work, however, integration of, e.g., BaSyx, is also feasible [11] potentially unlocking the possibility of an integrated shopfloor-building DT. Furthermore, our framework's modular architecture supports future extensibility and service integration, making it a versatile tool for various application contexts. This holistic approach marks a significant advance in engineering DTs for buildings, setting a precedent for future deployments in complex, multi-faceted environments.

Compared to prior approaches (cf. Section II) that often rely on proprietary systems or heavily manual integration methods, our framework leverages MDE principles to bridge the design-to-operation gap effectively. By using the BIM model as a central, structured data source, the framework supports a continuous chain from planning to operation. The use of BIM, in this respect, is critical not only for geometric and semantic modeling but also for uniquely identifying building components in the enhanced visualization environment offered by UE5. This integration is shown to support facility management tasks such as condition monitoring and remote control thus providing a foundation for maintenance planning, space management, and energy analysis [78].

D. Limitations and Future Work

While the initial evaluation in our Living Lab has underscored the efficiency and potential of our proposal under controlled conditions, several limitations remain that warrant further attention:

Living Lab Conditions The ODTE measurements were obtained under controlled network and system conditions that may not fully reflect the uncertainties encountered in real-world deployments. Future work should involve field studies in heterogeneous environments to evaluate performance under variable network latency, jitter, and potential state drift.

Requirements While the essential requirements for the targeted use cases (cf. Table III) have been addressed in our spike solution, real-world deployments would necessitate the incorporation of additional requirements. Notably, areas such as security aspects, access control, usability, and other operational considerations remain to be fully developed to support robust and scalable implementations in diverse building contexts.

IFC Support Currently, our implementation supports IFC 2x3. Although support for IFC 4x3 is planned, it has been postponed due to a known issue in Revit wherein property sets critical for twinning — such as photometric values — are not exported. Addressing this limitation is a key objective for subsequent iterations.

Extensibility and Integration Although our framework provides standardized extension points for integrating custom DT services, further work is needed to ensure seamless interoperability across additional IoT protocols and third-party analytics services. Continuous refinement of these modular interfaces will support broader deployment scenarios.

Visualization Platform The use of UE5 as a DT visualization platform offers significant benefits for immersive facility management. However, further research is required to optimize scalability and usability, ensuring that the visualization layer can support increasingly complex models and real-time data feeds in large-scale building environments.

Future work will address these limitations by field evaluations, enhancing IFC support, and refining the integration of custom services. By incorporating feedback from real-world deployments, we aim to solidify the practical relevance and robustness of our approach in supporting scalable and interoperable DTs in the AECO domain.

VI. Conclusion

Our work introduces a robust model-driven framework for deploying DTs in the AECO sector, effectively addressing the limitations of fragmented data and manual processes. By leveraging BIM as a semantic backbone and integrating middleware solutions like Eclipse Hono and Ditto, our approach facilitates real-time, bidirectional data exchange and interactive visualization through UE5. Our framework not only automates the deployment process, significantly reducing manual effort, but also supports modular, service-oriented extensions, enabling scalable and interoperable DT systems in built environments. The successful application in our Living Lab case study underscores its practical viability, with our proposal achieving near-ideal synchronization as measured by the ODTE metric.

The implications of our work are substantial for the AECO domain, paving the way for more efficient, data-driven building management and operations. By providing a interface for the seamless integration of DTs into existing workflows, our solution empowers stakeholders across the building life cycle to enhance decision-making, optimize resource usage, and improve the overall sustainability of infrastructure projects. The modularity and extensibility of our solution ensure that it can adapt to evolving technological and operational needs, making it an invaluable asset for future-proofing the built environment against the backdrop of rapid digital transformation.

REFERENCES

[1] Grieves, Michael and Vickers, John, "Digital twin: Mitigating unpredictable, undesirable emergent behavior in complex systems," *Transdisciplinary perspectives on complex systems: New findings and approaches*, pp. 85–113, 2017.

[2] Zech, Philipp and Clark, Tony and Breu, Ruth, "An Empirical Analysis of Digital Twin Adoption," in *58th Hawaiin Int. Conf. on System Sciences (HICSS'58)*, 2025, pp. 6253–6262.

[3] Sepasgozar, Samad M. E. and Hui, Felix Kin Peng and Shirowzhan, Sara and Foroozanfar, Mona and Yang, Liming and Aye, Lu, "Lean Practices Using Building Information Modeling (BIM) and Digital Twinning for Sustainable Construction," *Sustainability*, vol. 13, p. 161, 2020.

[4] Deng, Min and Menassa, Carol C. and Kamat, Vineet R., "From BIM to digital twins: a systematic review of the evolution of intelligent building representations in the AEC-FM industry," *Journal of Information Technology in Construction*, vol. 26, pp. 58–83, 2021.

[5] Broo, Didem Gürdür and Schooling, Jennifer, "Digital twins in infrastructure: definitions, current practices, challenges and strategies," *International Journal of Construction Management*, pp. 1–10, 2021.

[6] Michael, Judith and Blankenbach, Jörg and Derksen, Jan and Finklenburg, Berit and Fuentes, Raul and Gries, Thomas and Hendiani, Sepehr and Herlé, Stefan and Hesseler, Stefan and Kimm, Magdalena and Kirchhof, Jörg Christian and Rumpe, Bernhard and Schüttrumpf, Holger and Walther, Grit, "Integrating models of civil structures in digital twins: State-of-the-Art and challenges," *Journal of Infrastructure Intelligence and Resilience*, p. 100100, 2024.

[7] Hauer, Martin and Hammes, Sascha and Zech, Philipp and Geisler-Moroder, David and Plörer, Daniel and Miller, Josef and van Karsbergen, Vincent and Pfluger, Rainer, "Integrating digital twins with BIM for enhanced building control strategies: A systematic literature review focusing on daylight and artificial lighting systems," *Buildings*, vol. 14, no. 3, p. 805, 2024.

[8] Donkers, Alex JA and Yang, Dujuan and de Vries, Bauke and Baken, Nico HG, "Real-time building performance monitoring using semantic digital twins," in *9th Linked Data in Architecture and Construction Workshop, LDAC 2021*. CEUR-WS. org, 2021, pp. 55–66.

[9] Zech, Philipp and Hammes, Sascha and Goldin, Emanuele and Geisler-Moroder, David and Breu, Ruth and Pfluger, Rainer, "From BIM to Digital Twin: A transformation process through advanced control modeling and automated commissioning using daylight and artificial lighting as examples," *Energy and Buildings*, vol. 329, p. 115184, 2025.

[10] Lu, Qiuchen and Xie, Xiang and Parlikad, Ajith Kumar and Schooling, Jennifer Mary, "Digital twin-enabled anomaly detection for built asset monitoring in operation and maintenance," *Automation in Construction*, vol. 118, p. 103277, 2020.

[11] Zech, Philipp and Nardin, Claudio and Ristov, Sashko and Flora, Matthias and Breu, Ruth, "Digital-Twins-as-a-Service in Construction Engineering," in *2024 IEEE 20th International Conference on Automation Science and Engineering (CASE)*. IEEE, 2024, pp. 3004–3010.

[12] Zech, Philipp and Plörer, Daniel and Pfluger, Rainer and Breu, Ruth, "Scalable Building Energy Efficiency Balancing," in *2024 Annual Modeling and Simulation Conference (ANNSIM)*. IEEE, 2024, pp. 1–13.

[13] Zech, Philipp and Jäger, Alexandra and Schneiderbauer, Larissa and Exenberger, Hans and Fröch, Georg and Flora, Matthias, "Agile Construction Digital Twin Engineering," *Buildings*, vol. 15, no. 3, p. 386, 2025.

[14] Hammes, Sascha and Geisler-Moroder, David and Weninger, Johannes and Zech, Philipp and Pfluger, Rainer, "Market demands vs. scientific realities: A comparative analysis in the context of BIM-based and user-centred lighting control," *Developments in the Built Environment*, vol. 19, p. 100526, 2024.

[15] Eclipse Foundation, "Eclipse hono - connect, command & control iot devices," https://eclipse.dev/hono/, 2025, accessed: 2025-07-04.

[16] ——, "Eclipse ditto - iot middleware for digital twins," https://eclipse.dev/ditto/, 2025, accessed: 2025-07-04.

[17] Epic Games, "Unreal engine," https://www.unrealengine.com, 2025, accessed: 2025-07-04.

[18] Bellavista, Paolo and Bicocchi, Nicola and Fogli, Mattia and Giannelli, Carlo and Mamei, Marco and Picone, Marco, "Odte: A metric for digital twin entanglement," *IEEE Open Journal of the Communications Society*, 2024.

[19] University of Innsbruck, "TwinLight - BIM-based realisation of daylight and artificial lighting control systems," https://www.uibk.ac.at/en/energy-efficient-building/research/projects/twinlight/, 2025, accessed: 2025-07-04.

[20] ——, "BOREALIS - Self-learning building controls for a greener and healthier society," https://projekte.ffg.at/projekt/5121377, 2025, accessed: 2025-07-04.

[21] Zech, Philipp and Goldin, Emanuele and Hammes, Sascha and Geisler-Moroder, D and Pfluger, Rainer and Breu, Ruth, "Model-based auto-commissioning of building control systems," in *26th International Conference on Enterprise Information Systems (ICEIS 2024)*, 2024, pp. 121–128.

[22] Zhang, X. and Wang, Y. and Wang, J., "Digital twin application in the construction industry: A literature review," *Automation in Construction*, vol. 123, p. 103502, 2021.

[23] Li, H. and Hou, L. and Wang, X. and Guo, H., "Towards Civil Engineering 4.0: Concept, workflow and application of digital twins in civil engineering," *Automation in Construction*, vol. 135, p. 104114, 2022.

[24] Li, Y. and Wu, P. and Wang, J. and Wang, X., "Digital twin applications for overcoming construction supply chain management challenges: A review," *Automation in Construction*, vol. 140, p. 104262, 2023.

[25] Boje, C. and Guerriero, A. and Kubicki, S. and Rezgui, Y., "Towards a semantic Construction Digital Twin: Directions for future research," *Automation in Construction*, vol. 104, p. 102103, 2020.

[26] Wang, J. and Zhang, X. and Wang, Y., "Digital twin for product versus project lifecycles' development in manufacturing and construction: A comparative review," *The International Journal of Advanced Manufacturing Technology*, vol. 119, pp. 1–18, 2023.

[27] Ozturk, Gozde Basak, "Digital twin research in the AECO-FM industry," *Journal of Building Engineering*, vol. 40, p. 102730, 2021.

[28] Moshood, Taofeeq D and Rotimi, James OB and Shahzad, Wajiha and Bamgbade, JA, "Infrastructure digital twin technology: A new paradigm for future construction industry," *Technology in Society*, vol. 77, p. 102519, 2024.

[29] AlBalkhy, Wassim and Karmaoui, Dorra and Ducoulombier, Laure and Lafhaj, Zoubeir and Linner, Thomas, "Digital twins in the built environment: Definition, applications, and challenges," *Automation in Construction*, vol. 162, p. 105368, 2024.

[30] Gil, Santiago and Mikkelsen, Peter H and Gomes, Cláudio and Larsen, Peter G, "Survey on open-source digital twin frameworks–A case study approach," *Software: Practice and Experience*, vol. 54, no. 6, pp. 929–960, 2024.

[31] Eclipse Foundation, "Eclipse basyx - open source platform for industry 4.0," https://eclipse.dev/basyx/, 2025, accessed: 2025-07-04.

[32] Bentley Systems, "itwin.js - the library for developing infrastructure digital twin applications," https://www.itwinjs.org, 2025, accessed: 2025-07-04.

[33] D. Gerber, N. McGlashan, S. Breslav, and A. Khan, "Living lab digital twin: Case study of the development of a research-oriented digital twin in a leed platinum academic building," in *2025 Annual Modeling and Simulation Conference (ANNSIM'25)*. Society for Modeling & Simulation International, 2025, pp. 1–15.

[34] D.-G. J. Opoku, S. Perera, R. Osei-Kyei, M. Rashidi, K. Bamdad, and T. Famakinwa, "Digital twin for indoor condition monitoring in living labs: University library case study," *Automation in Construction*, vol. 157, p. 105188, 2024.

[35] Schmidt, Douglas C, "Model-driven Engineering," *Computer*, vol. 39, no. 2, p. 25, 2006.

[36] T. Stahl and M. Völter, *Model-driven software development: Technology, engineering, management*. Chichester: Wiley, 2006.

[37] Burgueño, Loli and Di Ruscio, Davide and Sahraoui, Houari and Wimmer, Manuel, "Automation in Model-Driven Engineering: A look back, and ahead," *ACM Transactions on Software Engineering and Methodology*, 2025.

[38] Teper, Thomas and Campo, Kelly X. and Eaton, Casey and Bhatia, Garima and Mesmer, Bryan, "Considerations for Implementation of Model-Based Systems Engineering in Different Sectors and System Types Based on Academic Literature," *Systems Engineering*, vol. 28, no. 1, pp. 3–21, 2025.

[39] M. Brambilla, J. Cabot, and M. Wimmer, *Model-Driven Software Engineering in Practice*. Cham: Springer International Publishing, 2017.

[40] Morales, Sergio and Clarisó, Robert and Cabot, Jordi, "Impromptu: a framework for model-driven prompt engineering," *Software and Systems Modeling*, 2025.

[41] J. Michael, L. Cleophas, S. Zschaler, T. Clark, B. Combemale, T. Godfrey, D. Khelladi, V. Kulkarni, D. Lehner, B. Rumpe, M. Wimmer, A. Wortmann, S. Ali, B. Barn, I. Barosan, N. Bencomo, F. Bordeleau,

G. Grossmann, G. Karsai, O. Kopp, B. Mitschang, P. Muñoz Ariza, A. Pierantonio, F. Polack, M. Riebisch, H. Schlingloff, M. Stumptner, A. Vallecillo, M. van den Brand, and H. Vangheluwe, "Model-Driven Engineering for Digital Twins: Opportunities and Challenges," *INCOSE Systems Engineering*, 2025.

[42] Kirchhof, Jörg Christian and Michael, Judith and Rumpe, Bernhard and Varga, Simon and Wortmann, Andreas, "Model-driven digital twin construction: synthesizing the integration of cyber-physical systems with their information systems," in *23rd ACM/IEEE Int. Conf. on Model Driven Engineering Languages and Systems*, 2020, pp. 90–101.

[43] Pfeiffer, Jérôme and Fuchß, Dominik and Kühn, Thomas and Liebhart, Robin and Neumann, Dirk and Neimöck, Christer and Seiler, Christian and Koziolek, Anne and Wortmann, Andreas, "Modeling Languages for Automotive Digital Twins: A Survey Among the German Automotive Industry," in *ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*, 2024, pp. 92–103.

[44] Vale, Sushant and Reddy, Sreedhar and Subramanian, Sivakumar and Chaudhuri, Subhrojyoti Roy and Nistala, Sri Harsha and Deodhar, Anirudh and Runkana, Venkataramana, "A Model-driven Approach for Knowledge-based Engineering of Industrial Digital Twins," in *2023 ACM/IEEE 26th International Conference on Model Driven Engineering Languages and Systems (MODELS)*. IEEE, 2023, pp. 13–23.

[45] Barat, Souvik and Kulkarni, Vinay and Clark, Tony and Barn, Balbir, "Digital twin as risk-free experimentation aid for techno-socio-economic systems," in *25th international conference on model driven engineering languages and systems*, 2022, pp. 66–75.

[46] Bordeleau, Francis and Combemale, Benoit and Eramo, Romina and Van Den Brand, Mark and Wimmer, Manuel, "Towards model-driven digital twin engineering: Current opportunities and future challenges," in *International conference on systems modelling and management*. Springer, 2020, pp. 43–54.

[47] D. Bano, J. Michael, B. Rumpe, S. Varga, and M. Weske, "Process-Aware Digital Twin Cockpit Synthesis from Event Logs," *Journal of Computer Languages (COLA)*, vol. 70, 2022.

[48] Lehner, Daniel and Zhang, Jingxi and Pfeiffer, Jérôme and Sint, Sabine and Splettstößer, Ann-Kathrin and Wimmer, Manuel and Wortmann, Andreas, "Model-driven engineering for digital twins: a systematic mapping study," *Software and Systems Modeling*, pp. 1–39, 2025.

[49] Abbasi, Faima and Brimont, Pierre and Pruski, Cedric and Sottet, Jean-Sébastien, "Understanding Semantic Drift in Model Driven Digital Twins," in *ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*, 2024, pp. 419–430.

[50] Kamburjan, Eduard and Bencomo, Nelly and Tapia Tarifa, Silvia Lizeth and Johnsen, Einar Broch, "Declarative Lifecycle Management in Digital Twins," in *ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*, 2024, pp. 353–363.

[51] Oakes, Bentley and Gomes, Claudio and Kamburjan, Eduard and Abbiati, Giuseppe and Ecem Bas, Elif and Engelsgaard, Sebastian, "Towards Ontological Service-Driven Engineering of Digital Twins," in *ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*, 2024, pp. 464–469.

[52] Giulianelli, Andrea and Burattini, Samuele and Ciortea, Andrei and Ricci, Alessandro, "Engineering Interoperable Ecosystems of Digital Twins: A Web-based Approach," in *ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*, 2024, pp. 476–485.

[53] Rathore, M Mazhar and Shah, Syed Attique and Shukla, Dhirendra and Bentafat, Elmahdi and Bakiras, Spiridon, "The role of ai, machine learning, and big data in digital twinning: A systematic literature review, challenges, and opportunities," *IEEE Access*, vol. 9, pp. 32 030–32 052, 2021.

[54] Skoury, Lior and Leder, Samuel and Menges, Achim and Wortmann, Thomas, "Digital twin architecture for the AEC Industry: A case study in collective robotic construction," in *ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*, 2024, pp. 413–418.

[55] Keskin, Basak and Salman, Baris and Koseoglu, Ozan, "Architecting a BIM-based digital twin platform for airport asset management: a model-based system engineering with SysML approach," *Journal of construction engineering and management*, vol. 148, no. 5, p. 04022020, 2022.

[56] Zech, Philipp and Jäger, Alexandra and Fröch, Georg and Pfluger, Rainer and Breu, Ruth, "Agile, continuous building energy modeling and simulation," *SIMULATION*, p. 00375497241251852, 2024.

[57] P. Zech, E. Goldin, C. Zallinger, S. Hammes, P. Pobitzer, J. Michael, and R. Breu, "An Ecosystem of DSMLs for Building Commissioning," in *28th ACM/IEEE International Conference on Model Driven Engineering Languages and Systems*. IEEE, 2025, accepted for publication.

[58] P. Zech, C. Kelter, S. Hammes, J. Michael, and R. Breu, "Meta-metamodel-Independent Model Transformations," in *51st Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. Springer, 2025, accepted for publication.

[59] Wieringa, Roel, *Design science methodology for information systems and software engineering*. Springer, 2014.

[60] M. Chowdhury, M. R. Hosseini, D. J. Edwards, I. Martek, and S. Shuchi, "Comprehensive analysis of bim adoption: From narrow focus to holistic understanding," *Automation in Construction*, vol. 160, p. 105301, 2024.

[61] Ž. Turk, "Ten questions concerning building information modelling," *Building and environment*, vol. 107, pp. 274–284, 2016.

[62] J. Liu, J. Liu, C. Zhuang, Z. Liu, and T. Miao, "Construction method of shop-floor digital twin based on mbse," *Journal of Manufacturing Systems*, vol. 60, pp. 93–118, 2021.

[63] B. Succar, W. Sher, and A. Williams, "An integrated approach to bim competency assessment, acquisition and application," *Automation in construction*, vol. 35, pp. 174–189, 2013.

[64] A. Thulgharia and O. Sumant. (2023) Building Information Modeling Market Size, Share, Competitive Landscape and Trend Analysis Report, by Component, by Deployment Mode, by Building Type, by Application, by End-User: Global Opportunity Analysis and Industry Forecast, 2022–2032. Allied Market Research. Report code A00919. [Online]. Available: https://www.alliedmarketresearch.com/building-information-modeling-market

[65] P. Zech, P. Burger, S. Hammes, D. Geisler-Moroder, and R. Breu, "Bimreason: Validating bim model correctness," *Bauphysik*, vol. 46, no. 6, pp. 332–339, 2024.

[66] Epic Games, "Unreal engine datasmith," https://www.unrealengine.com/en-US/datasmith, 2025, accessed: 2025-07-04.

[67] M. Laakso and A. Kiviniemi, "The ifc standard: A review of history, development, and standardization, information technology," *ITcon*, vol. 17, no. 9, 2012.

[68] The Kubernetes Authors, "Kubernetes," https://kubernetes.io, 2025, accessed: 2025-07-04.

[69] Apache Software Foundation, "Apache kafka," https://kafka.apache.org, 2025, accessed: 2025-07-04.

[70] The Helm Authors, "Helm - the kubernetes package manager," https://helm.sh, 2025, accessed: 2025-07-04.

[71] MongoDB, Inc., "Mongodb," https://www.mongodb.com, 2025, accessed: 2025-07-04.

[72] P. Zech, G. Fröch, and R. Breu, "A requirements study on model repositories for digital twins in construction engineering," in *International Conference on Cooperative Information Systems*. Springer, 2023, pp. 459–469.

[73] P. Zech, P. Pobitzer, G. Fröch, and R. Breu, "A Proposal for a Models-Meet-Data Repository For Digital Twins in Construction Engineering," in *2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-C)*. IEEE, 2024, pp. 111–118.

[74] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, 1994.

[75] M. Hossain, S. Leminen, and M. Westerlund, "A systematic review of living lab literature," *Journal of cleaner production*, vol. 213, pp. 976–988, 2019.

[76] S. Senoner and P. Zech, "Artifact for EDTConf'25 Submission: Model-driven Digital Twins for AECO," Jul. 2025. [Online]. Available: https://doi.org/10.5281/zenodo.15808203

[77] M. Batty, "Digital twins," *Environment and Planning B: Urban Analytics and City Science*, vol. 45, no. 5, pp. 817–820, 2018.

[78] R. Volk, J. Stengel, and F. Schultmann, "Building information modeling (bim) for existing buildings—literature review and future needs," *Automation in construction*, vol. 38, pp. 109–127, 2014.